Parallel and Distributed Computing

Assignment 5

Instructor: Dr. Saif Nalband

Course Code: UCS645

Introduction

This assignment explores key concepts in MPI-based parallel programming, including the distinction between blocking and non-blocking communication, the role of collective operations, and dynamic load balancing using a master-worker model.

Five different programs were implemented and executed using MPICH (Google Colab environment). Performance evaluation was carried out using `MPI_Wtime()` to analyze speedup and efficiency across multiple process counts (1, 2, 4, and 8).

Question 1: Parallel DAXPY

Objective

Implement the DAXPY operation:

X[i]=a·X[i]+Y[i]X[i] = a \cdot X[i] + Y[i]X[i]=a·X[i]+Y[i]

and evaluate performance improvements using parallel execution.

Implementation Highlights

- Vector size: 65,536
- Data distributed using `MPI_Scatter`
- Local computation performed on each process
- Results collected using `MPI_Gather`

- Execution time measured via `MPI_Wtime()`

Performance Summary

Processes	Time (s)	Speedup	Efficiency
1	0.003559	1.00	100%
2	0.002924	1.22	61%
4	0.004492	0.79	20%
8	0.004005	0.89	11%

Analysis

- Performance improves slightly when moving from 1 to 2 processes
- Further increase leads to performance degradation
- Communication overhead (Scatter/Gather) dominates computation
- Small problem size limits scalability

Conclusion

Parallelization works correctly but offers limited benefit for small input sizes. Larger datasets are required to achieve meaningful speedup.

Question 2: Broadcast Comparison

Objective

Compare manual broadcast (linear) with `MPI_Bcast`.

Implementation

- Array size: 10 million doubles
- **Part A:** Linear broadcast using `MPI_Send` / `MPI_Recv`
- **Part B:** Optimized broadcast using `MPI_Bcast`
- Execution time measured for both approaches

Performance Results

Processes	MyBcast (s)	MPI_Bcast (s)
2	0.081504	0.036116
4	0.432256	0.117063
8	3.055432	0.291714
16	6.282705	0.746867

Analysis

- Linear broadcast scales poorly (O(p))
- `MPI_Bcast` uses tree-based communication (~O(log p))
- Performance gap increases with process count

Conclusion

Optimized MPI collective operations significantly outperform manual implementations, especially at scale.

Question 3: Dot Product & Amdahl's Law

Objective

Compute dot product in parallel and analyze speedup limitations.

Key Points

- Data generated locally to reduce memory usage
- Multiplier distributed using `MPI_Bcast`
- Partial results combined using `MPI_Reduce`
- Problem size: 50 million elements

Performance Summary

Processes	Time (s)	Speedup	Efficiency
1	0.187807	1.00	100%
2	0.177762	1.06	53%
4	0.128404	1.46	36%
8	0.145019	1.29	16%

Analysis

- Speedup is not linear
- Communication overhead increases with processes
- Amdahl’s Law limits scalability

Conclusion

Parallelization improves execution time but efficiency decreases as process count increases due to communication and synchronization overhead.

Question 4: Prime Number Computation

Approach

- Master-worker (dynamic scheduling) model
- Workers request tasks dynamically
- Prime results returned to master

Observations

- Total primes found: 303 (up to 2000)
- Efficient load balancing achieved
- Minimal idle time due to dynamic allocation

Conclusion

Dynamic task distribution improves performance for irregular workloads and ensures better utilization of processes.

Question 5: Perfect Number Computation

Approach

- Master distributes numbers to workers
- Workers check perfect numbers and return results
- Termination handled via signal (-1)

Results

Perfect numbers up to 10,000:
6, 28, 496, 8128

Analysis

- Dynamic scheduling improves load distribution
- Use of `MPI_ANY_SOURCE` increases flexibility
- Efficient communication reduces idle time

Conclusion

The implementation successfully demonstrates parallel computation using MPI with effective load balancing.

Overall Conclusion

This assignment demonstrates multiple aspects of parallel computing:

- Parallelization improves performance when computation dominates communication
- Collective operations (`MPI_Bcast`, `MPI_Reduce`) are highly optimized

- Master-worker model is effective for dynamic workloads
- Scalability depends on problem size and communication overhead

---

Future Scope

- Use non-blocking communication (`MPI_Isend`, `MPI_Irecv`)
- Increase problem size for better scalability
- Explore hybrid parallelism (MPI + OpenMP)
- Optimize load balancing and communication strategies